

upSkill CAMPUS

INDUSTRIAL INTERNSHIP REPORT

BANKING INFORMATION SYSTEM

Python Internship Project

Prepared By

Vickey Raj

B.Tech – Computer Science & Engineering

IILM University, Greater Noida

Project Domain	Python
Project Type	Console-based Banking Application
Repository	upskillcampus
Code Link	TO BE UPDATED AFTER GITHUB UPLOAD
Report Link	TO BE UPDATED AFTER FIRST REPORT UPLOAD

TABLE OF CONTENTS

Section	Topic
1	Preface
2	Introduction
2.1	About the Internship
2.2	Objectives
2.3	Reference and Glossary
3	Problem Statement
4	Existing and Proposed Solution
4.1	Existing Solution
4.2	Proposed Solution
4.3	GitHub Submission Links
5	Proposed Design / Model
5.1	High Level Flow
5.2	Low Level Flow
5.3	Interfaces
6	Performance Test
6.1	Test Cases
6.2	Test Procedure
6.3	Performance Outcome
7	My Learnings
8	Future Work Scope

1 Preface

This industrial internship project was completed to gain practical experience in Python programming and project-oriented problem solving. The project selected for the final submission is a Banking Information System, implemented as a console-based Python application.

The application combines fundamental Python concepts into one practical program. It accepts user input, stores account records, updates balances, records transactions, and persists information in a JSON file.

Project planning:

- Understand the banking use case and define the main operations.
- Implement account creation and data storage.
- Implement deposit, withdrawal and balance checking.
- Add transaction history and file persistence.
- Test valid and invalid inputs.
- Document the project and submit source code and report through GitHub.

2 Introduction

2.1 About the Internship

The supplied sample report presents the internship as a practical learning and project-development experience. It describes a structured program in which learners work on an assigned problem statement and document their implementation, testing and learning outcomes. ■filecite■turn0file0■L8-L15■

2.2 Objectives

- Develop practical Python programming skills.
- Apply programming concepts to a complete project.
- Improve logical thinking and problem-solving ability.
- Practice debugging, validation and testing.
- Learn to document and submit a project professionally.

2.3 Reference and Glossary

Term	Meaning
Python	Programming language used to implement the project.
List	Collection used to store multiple values.
append()	Method used to add an item to a list.
Function	Reusable block of code defined with def.
File handling	Reading from and writing information to files.
JSON	Text-based format used here to persist account data.
GitHub	Platform used to store and share the source code and report.

3 Problem Statement

Basic banking operations involve maintaining customer information, account balances and transaction records. For a learning environment, a small console application can demonstrate how these operations can be represented programmatically.

The problem addressed by this project is to create a simple Banking Information System that allows a user to create an account and perform basic account operations while keeping the information available between program runs.

Functional requirements:

- Create a new account with customer name, account number and opening balance.
- Prevent duplicate account numbers.
- Deposit money into an existing account.
- Withdraw money only when sufficient balance is available.
- Display the current balance.
- Display transaction history.
- Store account information in a file for later use.

Scope limitation: This is an educational application and is not designed for real banking, authentication, encryption, regulatory compliance or handling real financial information.

4 Existing and Proposed Solution

4.1 Existing Solution

A basic manual approach would require keeping account information and transaction records separately and updating them whenever an operation is performed. Such an approach can be repetitive and is more difficult to validate consistently.

4.2 Proposed Solution

The proposed solution is a menu-driven Python application. Functions are used to separate operations such as account creation, deposit, withdrawal, balance checking and transaction-history display. Account records are maintained as structured Python data and saved to a JSON file.

- Menu-driven interaction through input().
- Reusable functions defined using def.
- Lists/dictionaries for structured records.
- append() for adding transaction records.
- JSON file handling for persistence.
- Validation for empty fields, invalid numbers and insufficient balance.

4.3 GitHub Submission Links

Item	Final Link
Repository	https://github.com/YOUR_USERNAME/upskillcampus
Code	https://github.com/YOUR_USERNAME/upskillcampus/blob/main/banking_information_system.py
Report	https://github.com/YOUR_USERNAME/upskillcampus/blob/main/Vickey_BankingInformationSystem_Final_Report.pdf

Replace YOUR_USERNAME with the actual GitHub username after creating the repository. The report must be uploaded once before its own GitHub URL can be inserted, then the updated PDF should replace the earlier PDF.

5 Proposed Design / Model

5.1 High Level Flow

Step	Process
1	Start application
2	Load saved account data from accounts.json
3	Display main menu
4	Accept user's choice
5	Call the corresponding function
6	Validate input and update account data
7	Save changes to accounts.json
8	Display result and return to menu
9	Exit when the user selects option 6

5.2 Low Level Flow

- `load_accounts()` reads the JSON file.
- `find_account()` searches the list for an account number.
- `create_account()` collects details and uses `append()` to add a new account.
- `deposit_money()` increases the balance and appends a transaction.
- `withdraw_money()` validates the amount and available balance before reducing the balance.
- `check_balance()` displays the current account information.
- `show_transactions()` displays stored transaction records.
- `save_accounts()` writes updated data back to the JSON file.

5.3 Interfaces

The project uses a command-line interface. Users interact through numbered menu options and text prompts. The persistent data interface is the `accounts.json` file, which stores account records and transaction histories.

6 Performance Test

The project was designed to be tested through functional test cases covering normal operations and common invalid inputs. The purpose is to verify that each feature behaves as expected and that account data remains consistent.

6.1 Test Cases

ID	Operation	Condition	Expected Result
TC-01	Create account	Valid name, unique account no., opening balance	Account created
TC-02	Duplicate account	Existing account number	Creation rejected
TC-03	Deposit	Valid positive amount	Balance increases
TC-04	Withdraw	Amount less than/equal to balance	Balance decreases
TC-05	Insufficient balance	Withdrawal greater than balance	Withdrawal rejected
TC-06	Invalid amount	Text/non-numeric amount	Error handled
TC-07	Balance	Existing account number	Correct balance displayed
TC-08	History	Existing account number	Transactions displayed
TC-09	Persistence	Restart program after saving	Saved accounts loaded

6.2 Test Procedure

- Run the program using Python 3.
- Execute each menu operation with valid and invalid inputs.
- Compare actual output with expected behavior.
- Check that account data is written to and read from accounts.json.
- Repeat operations after restarting the program to verify persistence.

6.3 Performance Outcome

The implementation is intended for small educational datasets and provides immediate console responses for individual operations. For a real banking environment, a database, authentication, encryption, auditing, concurrency controls and stronger security would be required.

7 My Learnings

- Learned how to structure a Python program into reusable functions using `def`.
- Practiced taking user input using `input()`.
- Improved understanding of lists and dictionaries.
- Used `append()` to add accounts and transaction records.
- Learned the basics of reading and writing persistent data using files.
- Practiced validation and exception handling for user input.
- Improved understanding of loops, conditions and program flow.
- Learned how several basic concepts can be combined into a complete project.
- Gained practical experience in organizing source code and preparing a GitHub submission.

The project also reinforced the importance of testing edge cases instead of checking only the normal path.

8 Future Work Scope

- Add login and role-based authentication.
- Replace JSON storage with a relational database.
- Add account deletion and account-update features.
- Add fund-transfer functionality.
- Add stronger validation and error logging.
- Create a graphical or web-based interface.
- Add automated unit tests.
- Use encryption and secure credential handling for a production-oriented implementation.
- Deploy the application as a web service if required.

These improvements would extend the educational prototype into a more complete software application, while requiring additional security, database and software-engineering considerations.

FINAL SUBMISSION CHECKLIST

- Repository name: upskillcampus.
- Upload banking_information_system.py.
- Upload Vickey_BankingInformationSystem_Final_Report.pdf.
- Upload README.md and supporting files if required.
- Copy the GitHub URL of the code file.
- Copy the GitHub URL of the report file.
- Insert both links into the report.
- Replace the first uploaded PDF with the updated PDF containing its own GitHub link.
- Verify the repository and both file links before submission.

The report structure follows the supplied sample's organization and its requirement to include code/report GitHub links. ■filecite■turn0file0■L150-L156■